Same Signal, Opposite Meaning: Why Adaptive Compute Fails Across Environments

Anonymous

Abstract

Large language model (LLM) agents benefit substantially from test-time compute, but applying expensive optimization to every decision step is wasteful. Exist gating methods address this by using uncertainty signals to decide when extra compute is worthwhile. However, these methods share an implicit assumption: a fixed mapping from signal value to compute benefit. We show that this assumption is wrong. Across eight diverse agent environments and three model backbones, we find that the signal–utility direction *reverses*: the same signal value that predicts rollout benefit in one environment predicts rollout harm in another. We trace this to a conflation of two uncertainty sources, *information poverty* and *decision difficulty*, and formalize it via a two-source model grounded in Simpson's paradox, where subgroup-level trends reverse upon aggregation. We prove that direction discovery is a necessary condition for non-negative value of computation across environments. Motivated by this result, we propose EAAG (Environment-Aware Adaptive Gating), which combines exploration, LLM-based reasoning, and sparse direction learning to discover environment-specific gating patterns. Across eight environments, EAAG outperforms all fixed-direction baselines with zero per-step overhead, and its adaptive gating behavior emerges from direction learning alone.

1 Introduction

Large language models (LLMs) have rapidly advanced across a wide range of tasks, from question answering and code generation to web navigation and embodied decision making (Yao et al., 2023b; Shinn et al., 2023; Zhou et al., 2024). However, many tasks remain beyond the reach of a single forward pass: complex reasoning, multi-step planning, and open-ended exploration often require the model to evaluate alternatives, verify intermediate results, or search over candidate actions (Yao et al., 2023a; Hao et al., 2023; Snell et al., 2024). This has motivated *test-time compute*, where additional computation at inference time improves decision quality. Across multiple benchmarks, test-time optimization allows smaller models to match or exceed the performance of larger ones at lower total cost (Snell et al., 2024; Wang et al., 2026).

However, test-time compute is expensive, incurring substantial per-step overhead (e.g., requiring multiple rollouts or candidate evaluations at each decision point) (Yao et al., 2023a; Zhou et al., 2024). As agents are deployed across diverse settings, applying this overhead uniformly becomes wasteful when only a fraction of steps benefit. This has motivated *adaptive gating*: using uncertainty signals to decide *when* extra compute is worthwhile. Across diverse mechanisms—entropy thresholding (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026), calibrated confidence (Muennighoff et al., 2025; Han et al., 2025), vote disagreement (Lee et al., 2026), and reinforcement learning (Zhang et al., 2025; Fang et al., 2025)—adaptive gating consistently reduces cost while preserving or improving task performance within each method's evaluation setting.

Methods calibrated for one task type frequently fail when transferred to others (Tao et al., 2025; Heo et al., 2025). Vote-based gating designed for web agents degrades on fact-verification tasks; calibration-based methods that perform well on QA degrade in code generation. In several cases, adaptive gating *actively harms* performance compared to never invoking the optimizer. This raises a central question: *do uncertainty signals have a universal relationship with compute value, or does this relationship depend on the environment?*

We conduct systematic measurement across eight agent environments spanning QA (Yang et al., 2018), code generation (Hendrycks et al.,

2021b), web navigation (Hendrycks et al., 2021a), fact verification (Thorne et al., 2018), and text games (Jansen and Côté, 2023), using three model backbones. The signal–utility direction *reverses* across environments: the same signal that predicts rollout benefit in code generation predicts rollout harm in fact verification, and can further reverse across model backbones within the same environment. Beyond direction, the most informative signal itself varies across environments, with structural features dominating in some while entropy carries no information.

We trace this reversal to a conflation of two uncertainty sources. *Information poverty* arises when the agent lacks sufficient data; here rollouts from the same model cannot compensate, and high entropy predicts rollout harm. *Decision difficulty* arises when the agent faces multiple viable paths; here rollouts explore alternatives productively, and high entropy predicts rollout benefit. Environments differ in the proportion of these two state types, causing the same signal to carry opposite meaning. This is an instance of Simpson's paradox (Simpson, 1951; Wang et al., 2023) in the signal–utility space. We prove that this has a sharp consequence: no fixed-direction gate can guarantee non-negative value of computation across environments with opposing directions (Proposition 1). Once the direction is wrong, more precise calibration makes performance *worse*, because the gate more precisely targets harmful states.

Motivated by this analysis, we propose **EAAG** (**E**nvironment-**A**ware **A**daptive **G**ating), which discovers environment-specific gating patterns through randomized *exploration*, LLM-based *reasoning* about exploration data, and sparse *learning* of signal direction and importance via LASSO. Since direction is the primary determinant of gate quality, not model capacity, EAAG pairs a rich discovery process with a deliberately simple gate that adds zero per-step deployment cost. EAAG outperforms all fixed-direction baselines across eight environments and three LLM backbones.

Our main contributions are as follows:

- **Empirical finding.** Signal–utility direction reverses across environments and model backbones, and the most informative signal itself differs across environments. All fixed-direction methods fail in at least two of eight environments.
- **Theoretical explanation.** A two-source uncertainty model grounded in Simpson's paradox explains when and why direction reverses. Direction discovery is provably necessary for cross-environment non-negative value of computation.
- **Adaptive gating method.** EAAG combines exploration, LLM-based reasoning, and sparse direction learning to discover environment-specific gating patterns with zero per-step deployment cost.

2 Related Work

Adaptive test-time compute. Adaptive gating methods reduce the overhead of test-time optimization by invoking it only when needed. In the *reasoning* setting, signal-based methods use token-level confidence or entropy to decide when to extend thinking (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). RL-based methods learn think/no-think policies through reinforcement learning (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025), though these require per-environment training and provide no interpretable direction signal. Hidden-state probing methods (Liu et al., 2025; Zhu et al., 2025) train lightweight classifiers on internal representations to estimate task difficulty, assuming a universal difficulty-to-compute mapping. In the *agent* setting, vote-based methods (Lee et al., 2026) sample multiple candidate actions and trigger evaluation when disagreement is high, and Paglieri et al. (2025) learn when to invoke a planner at the episode level. The vast majority of adaptive compute research operates on homogeneous reasoning benchmarks (MATH, GSM8K, GPQA), where signal direction happens to be consistent across tasks. Our work is the first to systematically study signal semantics across heterogeneous *agent* environments, where we find that the direction reverses. A detailed per-method comparison is provided in Appendix A.

Test-time scaling and search. Test-time scaling (Snell et al., 2024) studies compute-optimal allocation at the problem level, showing that adaptive allocation outperforms uniform allocation by up to 4× in efficiency. This line of work focuses on *how much* compute to allocate given a fixed difficulty signal, whereas we question the signal itself. Search methods such as Tree of Thoughts (Yao et al., 2023a), LATS (Zhou et al., 2024), RAP (Hao et al., 2023), and FLARE (Wang et al., 2026) improve the quality of the search pro-

cess through tree search, value propagation, and lookahead evaluation. These are complementary to our work: they improve *what* the optimizer does, while we control *when* to invoke it. Our gating framework can be composed with any such optimizer.

Uncertainty estimation and metareasoning. Recent work on LLM uncertainty (Tao et al., 2025; Heo et al., 2025) observes that calibration quality varies substantially across task types, providing converging evidence that uncertainty semantics are domain-dependent. Abbasi Yadkori et al. (2024) propose decoupling epistemic and aleatoric uncertainty in LLM outputs, a distinction conceptually related to our Type I/Type D decomposition. From a decision-theoretic perspective, rational metareasoning (Russell and Wefald, 1991; Sabbata et al., 2024) formalizes the value of computation (VOC) as a guide for allocating cognitive resources. Our work extends this framework by showing that VOC can be *negative* in the agent setting when the evaluator and executor are the same model, and that direction discovery is a necessary condition for non-negative VOC across environments.

3 Why Fixed-Direction Gating Fails

We test the fixed-direction assumption by measuring signal–utility correlations across diverse environments. If the assumption holds, correlations should be consistent; if it fails, the failure mode will guide method design.

We study LLM-based agents in multi-step interactive environments. At each step t, the agent observes state s_t, selects action a_t, and transitions to s_{t+1}. An environment-specific *test-time optimizer* T (e.g., rollout evaluation, K-variant sampling) can be invoked at any step to potentially improve action quality. We define the *optimizer utility* as:

$$U(T, s) = \mathbb{E}[R(\tau) \mid a{=}T(s)] - \mathbb{E}[R(\tau) \mid a{=}\pi(s)] \quad (1)$$

where $R(\tau)$ is the episode return. $U > 0$ indicates the optimizer improves the outcome; $U \leq 0$ indicates it is wasteful or harmful. The *adaptive gating problem* is to learn a gate $g : \mathcal{S} \to \{0, 1\}$ that triggers T only when $U > 0$.

3.1 Empirical Landscape

[FIGURE NEEDED: Figure 1.] Signal–utility scatter plots for 4 representative environments (FEVER, APPS Interview, HotpotQA, WebShop). Each panel: x-axis = token entropy, y-axis = optimizer utility U. Show the regression line with positive/negative slope to visually demonstrate direction reversal. Similar to FLARE Figure 1 (4-panel grid). This is the paper's most important figure – the reader should SEE the reversal.

Table 1: Signal–utility correlations across 8 environments and 3 backbones. ρ: Spearman correlation of token entropy with optimizer utility U. *: significant at $p{<}0.05$. Direction varies across *both* environments and model backbones.

Environment	Qwen3-4B	Phi-3.5	Llama-3.1	Consistent
HotpotQA	-0.327^*	$+0.184^*$	-0.346^*	×
APPS Intro	$+0.144^*$	-0.129^*	-0.242^*	×
WebShop	$+0.133^*$	$+0.335^*$	$+0.287^*$	✓
FEVER	-0.119^*	-0.156^*	$+0.428^*$	×
APPS Intv.	$+0.317^*$	-0.024	-0.249^*	×
TWExpress	-0.290^*	$+0.000$	$+0.000$	—
CRUXEval	-0.048	-0.065	-0.045	—
Plancraft	-0.016	$+0.000$	-0.176^*	—

Observation 1: The signal–utility direction reverses across environments *and* model backbones. Table 1 reports Spearman correlations between token entropy and optimizer utility across 8 environments and 3 model backbones. Two patterns stand out. First, entropy correlates *negatively* with utility in fact-verification environments (FEVER, HotpotQA) but *positively* in code generation (APPS Interview), with near-zero correlation in mixed environments (APPS Intro, WebShop). Second, the *same signal in the same environment* can reverse across backbones: among the 5 environments with at least two significant correlations, 4 show inconsistent signs across models. Structural signals like `step_count` are more stable across backbones, while entropy is unpredictable without per-environment, per-model measurement.

[FIGURE NEEDED: Figure 2.] Feature importance heatmap. Rows = 8 environments, Columns = features (step_count, token_entropy, state_length, num_available_actions, LLM-generated features). Cell color = LASSO coefficient magnitude, with sign indicated by +/-. Shows that different environments select completely different feature subsets. Similar to a feature-attribution matrix.

Observation 2: The identity of the most informative signal varies across environments. Sparse feature selection across environments (Fig-

ure **??**) reveals that different environments rely on entirely different signals: `step_count` is informative in 6/7 environments, but WebShop relies on `num_available_actions` and task-specific features instead. No single signal is universally informative.

[FIGURE NEEDED: Figure 3.] AUC hierarchy bar chart. 3 grouped bars per environment: single-entropy AUC (0.50), multi-signal logistic AUC (0.83), hidden-state probe AUC (0.90). Clearly shows the information gap between single-signal and multi-signal gates. 4 environments side by side (HotpotQA, APPS Intro, WebShop, FEVER).

Observation 3: Even the best single signal is insufficient. Beyond signal identity, even the *most* informative single signal per environment carries near-zero predictive power. AUC analysis across 4 environments (Figure **??**) reveals a clear hierarchy: single token entropy achieves AUC≈0.50 (indistinguishable from chance), whereas combining multiple signals reaches AUC≈0.83, and hidden-state probes reach AUC≈0.90. Predicting rollout value requires multiple signals, not a single scalar.

Observation 4: All fixed-direction methods fail systematically. When evaluated across all 8 environments, every fixed-direction baseline underperforms the no-trigger baseline in at least 2 environments (§5.2). Most strikingly, CATTS achieves 34.2% on FEVER, *below* the 37.0% no-trigger baseline, demonstrating that wrong-direction gating is actively harmful.

3.2 Why Does Direction Reverse? A Two-Source Model

The four observations above establish *that* fixed-direction gating fails; we now ask *why*. We propose a model that explains direction reversal through a single mechanism.

Intuition. Consider a web-shopping agent choosing between products. When it hesitates (high entropy) because multiple items match the query, exploring alternatives via rollouts is valuable, since the agent has adequate information but faces a genuine choice. Now consider a fact-verification agent that hesitates because the retrieved evidence is ambiguous. Here, high entropy reflects *missing information*, not optionality: rollouts from the same model with the same evidence will not resolve the ambiguity. In the first case, entropy positively predicts rollout utility; in the second, it predicts rollout *futility*. The same numerical signal carries opposite meaning because the *source* of uncertainty differs. We formalize this intuition below.

Model. We model the signal–utility direction reversal as arising from two sources of uncertainty at each decision step:

- **Type I (information poverty):** The agent lacks sufficient information. High entropy reflects confusion, not optionality. Rollouts from the same model cannot compensate for missing information, so utility is *negatively* related to entropy: $U_I(s) \sim -\alpha \cdot H(s) + \varepsilon_I,\ \alpha > 0$.
- **Type D (decision difficulty):** The agent has adequate information but faces multiple viable paths. High entropy reflects optionality. Rollouts can exploit this diversity, so utility is *positively* related to entropy: $U_D(s) \sim +\beta \cdot H(s) + \varepsilon_D,\ \beta > 0$.

Let $p_I(\mathcal{E})$ denote the fraction of Type I states in environment $\mathcal{E}$. The marginal correlation becomes (full derivation in Appendix D):

$$\rho(\mathcal{E}) \ \approx\ \beta - (\alpha + \beta)\, p_I(\mathcal{E}) \tag{2}$$

Direction reversal occurs at the critical threshold $p_I^* = \beta/(\alpha + \beta)$: environments with $p_I > p_I^*$ exhibit negative ρ (Type I dominated), while those with $p_I < p_I^*$ exhibit positive ρ (Type D dominated). Near p_I^*, the two effects cancel and entropy carries negligible signal ($\rho \approx 0$).

Environment mapping. Table 2 maps each evaluation environment to its dominant uncertainty type based on task structure. This mapping is primarily determined by the environment's information structure, but multi-backbone experiments (Appendix E) reveal that the model backbone modulates the effective p_I: different models may process the same environment with different balances of Type I vs Type D uncertainty, causing entropy ρ to flip sign (e.g., FEVER: $\rho{=}-0.156$ on Phi-3.5 but $\rho{=}+0.428$ on Llama-3.1). Structural signals like `step_count` are more robust across backbones.

The mapping confirms the model's core prediction: *the sign of the entropy–utility correlation is determined by which uncertainty source dominates*, not by properties of the entropy signal itself. Multi-backbone experiments further show that different models shift the effective balance between

Table 2: Environment-to-type mapping. The dominant uncertainty type is shaped by the environment's information structure and modulated by the model backbone. Predicted ρ direction follows from Eq. (2); observed values are from Qwen3-4B. Multi-backbone ρ values differ (Table 1).

Environment	Dominant Type	Pred. ρ	Obs. ρ	Rationale
HotpotQA	Type I (info-poverty)	$-$	-0.041	Requires retrieving specific evidence
FEVER	Type I (info-poverty)	$-$	-0.119	Decides on finding supporting facts
TWExpress	Type I (info-poverty)	$-$	-0.290	Straightforward, long info-gathering
APPS Intv.	Type D (decision-diff)	$+$	$+0.317$	Multiple valid solution strategies
APPS Intro	Mixed ($p_I \approx p_I^*$)	≈ 0	$+0.012$	Mix of info-gathering & code design
WebShop	Mixed (entropy weak)	≈ 0	-0.019	Choice-dominated but entropy weak
CRUXEval	Weak signal	≈ 0	-0.048	Short trajectories, little variation
Plancraft	Weak signal	≈ 0	-0.016	Sequential decision, tool-use overhead

Table 3: Method classification by adaptive compute components. All prior methods share {Single signal, Fixed direction, Problem-level}. EAAG is the only method with {Multi-signal, Learned direction, Step-level, Environment-aware}.

Method	Signal	Direction	Granularity	Env-Aware
CaTS†	Single	Fixed	Problem	✗
EAAC†	Single	Fixed	Problem	✗
CoRe†	Single	Fixed	Problem	✗
ICAPS	Single	Fixed	Problem	✗
AUQ	Single	Fixed	Problem	✗
EAAG	**Multi (auto)**	**Learned**	**Step**	✓

Type I and Type D uncertainty in the same environment, making direction unpredictable from environment structure alone, reinforcing why adaptive direction discovery (rather than environment-level heuristics) is essential.

Theoretical grounding. This direction reversal is an instance of Simpson's paradox (Simpson, 1951; Pearl, 2022): aggregating heterogeneous subpopulations with opposing within-group trends reverses the aggregate correlation. Our two-source decomposition draws on the epistemic/aleatoric distinction (Der Kiureghian and Ditlevsen, 2009), adapted to the meta-decision setting: Type I states exhibit epistemic-like uncertainty (information deficit), while Type D states exhibit aleatoric-like diversity (multiple viable paths).

Proposition 1 (Necessity of Direction Discovery)**.** *Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions: $d^*(\mathcal{E}_1) = +1$ and $d^*(\mathcal{E}_2) = -1$. Then no fixed-direction gate g_d can simultaneously satisfy $\mathrm{SR}(g_d, \mathcal{E}_1) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_1)$ and $\mathrm{SR}(g_d, \mathcal{E}_2) \geq \mathrm{SR}(\mathrm{base}, \mathcal{E}_2)$. That is, direction discovery is a* necessary *condition for cross-environment non-negative value of computation.*

The intuition is straightforward: a gate with $d = +1$ triggers on high-signal states, which in a Type I environment correspond to information poverty where rollouts are harmful. The symmetric argument holds for $d = -1$ in Type D environments. Since no single direction satisfies both, direction discovery is necessary. The full proof is provided in Appendix C.

3.3 Implications for Method Design

The four observations and the necessity result above jointly establish three requirements for cross-environment adaptive compute. First, the gate must use *multiple signals*, since no single signal is universally informative (Obs. 2–3). Second, the gate must discover the *direction* of each signal per environment, since fixed direction is provably insufficient (Prop. 1). Third, the gate must adapt to each *environment's* information structure, since the dominant uncertainty type varies across tasks (Obs. 1, Table 2). The Two-Source Model further yields testable predictions regarding temporal dynamics, cross-environment consistency, and signal identity alignment, which we verify empirically in §5.4. We now present a method that satisfies all three requirements.

4 Method: Environment-Aware Adaptive Gating

Table 3 summarizes how existing methods fail to satisfy the three requirements identified in §3.3. We present EAAG (Environment-Aware Adaptive Gating), which satisfies all three by discovering both signal identity and direction from online interaction data.

Design rationale. Since direction dominates gate quality, not model capacity (§5.3, Table 5), EAAG invests complexity in the discovery process rather than the gate itself. An MLP gate with the wrong direction falls below the no-trigger baseline, while a logistic gate with the correct direction Pareto-dominates all baselines. The gate is therefore a logistic function; the discovery pipeline (exploration + LLM reasoning + LASSO) identifies which signals matter and in which direction.

[FIGURE NEEDED: Figure 4.] EAAG method overview diagram. Three-phase pipeline: Explore (randomized gating, collect D) → Reason (LLM analyzes D, generates environment profile + feature hypotheses) → Learn (LASSO selects features, trains logistic gate with signed weights). Show arrows from Obs 1-3 in §3 to each design choice. Similar to FLARE's method section which has equations inline but a clear conceptual flow. Consider a horizontal pipeline diagram.

EAAG operates in three phases (Figure **??**): exploration, reasoning, and learning.

4.1 Exploration

The agent explores a new environment with $N_{\text{explore}} = 50$ episodes using randomized gating ($\varepsilon = 0.5$): at each step, the optimizer T is invoked with probability 0.5 regardless of state. This produces a dataset $\mathcal{D} = \{(s_t^{(i)}, U_t^{(i)})\}$ of state–utility pairs. The randomized design ensures unbiased estimation of U across the full signal range, a requirement that targeted exploration would violate by conditioning on the signal whose semantics are unknown.

4.2 Feature Generation

An LLM analyzes $\mathcal{D}$ to identify patterns distinguishing useful from wasteful rollouts. It produces an *environment profile* describing the task structure and *feature hypotheses* proposing task-specific signals to extract. For example, in WebShop the LLM identifies that rollouts help during product comparison but not during search, proposing features such as `price_mentioned` and `action_is_click`. These complement a set of universal features (step count, token entropy, state length) available across all environments. The LLM's primary value is generalizability: it enables zero-shot feature generation for unseen environments where task-specific signals cannot be anticipated by universal features alone.

4.3 Sparse Direction Learning

LASSO selects the most relevant features from the union of universal and LLM-generated candidates, then trains a logistic regression gate:

$$g(s) = \mathbf{1}\Big[\sigma(\mathbf{w}^\top \phi(s) + b) > \tau\Big] \tag{3}$$

where $\phi(s)$ is the selected feature vector and $\mathbf{w}$ encodes both direction (sign) and importance (magnitude) for each signal. The sign of each w_i directly implements direction discovery: $w_i > 0$ triggers on high signal values (Type D semantics), $w_i < 0$ triggers on low values (Type I semantics), and $w_i = 0$ indicates an uninformative signal (LASSO selection). Training completes in <1 s on CPU. At deployment, the gate adds zero per-step cost.

4.4 Online Adaptation

During deployment, EAAG optionally continues collecting data via ε-greedy exploration ($\varepsilon : 0.1 \to 0$) and retrains the gate periodically, enabling adaptation to distributional shift.

5 Experiments

5.1 Experimental Setup

We evaluate EAAG along three axes: (1) Does environment-aware gating Pareto-dominate fixed-direction baselines across diverse environments? (§5.2) (2) Which components (direction learning, multi-signal features, LLM reasoning) drive the gains? (§5.3) (3) Do the Two-Source Model's predictions hold empirically? (§5.4)

Environments. We evaluate on 8 environments spanning QA, code generation, web navigation, fact verification, text games, and crafting planning (Table 4): HotpotQA, APPS Intro, WebShop, FEVER, TWExpress, Plancraft, APPS Interview, and CRUXEval. These environments are selected to cover the full range of the Two-Source Model, from pure Type I (FEVER) through mixed ($\approx p_I^*$, APPS Intro) to Type D (APPS Interview), including extreme rollout properties (TWExpress: rollout-safe; Plancraft: rollout-harmful).

Baselines. All methods share the same agent and optimizer T; we compare only the gate decision. (1) *Bounds*: base_only, always_trigger, oracle. (2) *Fixed-direction CB*: CaTS, SEAG, CoRefine, CATTS, AUQ, s1_budget, all assuming high uncertainty → trigger. (3) *Ablation*: BSW (intentionally reversed direction). (4) *Ours*: EAAG. Full environment specifications and hyperparameter details are provided in Appendix B.

Metrics. SR (success rate, ↑) and Cost (rollouts per episode during deployment, ↓). A method Pareto-dominates another if SR $\geq$ and Cost $\leq$ with at least one strict inequality. Cost measures only the *deployment* rollout budget; calibration or exploration overhead is excluded for all methods, ensuring a fair comparison of runtime efficiency.

Table 4: Environment setup. Base/Always: SR without/with optimizer. Δ: rollout headroom.

Environment	Base SR	Always SR	Δ	Optimizer T
HotpotQA	49.0%	97.0%	+48.0	Per-action eval
APPS Intro	58.5%	64.5%	+6.0	K-variant sampling
WebShop	7.2%	43.0%	+35.8	LLM-Propose 1
FEVER	37.0%	99.8%	+62.8	Per-action eval
TWExpress	67.5%	99.3%	+31.8	Per-action eval
Plancraft	29.8%	22.8%	−7.0	Per-action eval
APPS Interview	60.5%	79.5%	+19.0	K-variant sampling
CRUXEval	85.0%	99.5%	+14.5	K-variant sampling

5.2 Main Results

EAAG outperforms fixed-direction baselines across environments and backbones. As shown in Table **??**, EAAG consistently outperforms all fixed-direction baselines across 8 environments (Table **??**). The two exceptions occur in ceiling-effect environments where EAAG already approaches the oracle. EAAG Pareto-dominates the strongest calibration-based baseline in most shared environments while requiring no calibration data. Statistical significance details are in Appendix F.

Importantly, EAAG's behavior is *qualitatively different* from baselines: it does not simply improve numbers but exhibits environment-adaptive gating strategies, as we analyze below.

[DATA NEEDED: Table 3 (tab:main).] Main results table. Full method × environment grid. Rows = methods (base_only, always_trigger, oracle, CaTS, SEAG, CoRefine, CATTS, AUQ, s1_budget, BSW, EAAG). Columns = 8 environments, each with SR(%) and Cost (ro/ep). Bold best SR per environment. † marks Phase-1-dependent methods. Similar to FLARE Table 1 (benchmark × method × backbone grid).

[FIGURE NEEDED: Figure 5.] Pareto frontier plot. x-axis = Cost (rollouts/episode), y-axis = SR(%). One panel per environment (or 4 representative ones). Plot each method as a point. EAAG should be on or near the Pareto frontier. Similar to FLARE Figure 3 (performance-budget trade-off curves).

Cross-environment patterns. The pattern of results aligns with the Two-Source Model's predictions along three axes. First, the gap between EAAG and fixed-direction baselines is largest where direction mismatch is most severe: in Type I environments, all "high uncertainty → trigger" baselines degrade below the no-trigger baseline because the assumed direction is wrong, and no amount of calibration rescues them. Second, EAAG's advantage is smallest where direction matters least: in high-headroom environments where even imprecise gating works, most methods approach the oracle. Third, EAAG adapts not only direction but *magnitude* of gating to the environment's rollout value structure. In rollout-safe environments (TWExpress, Δ=+31.8 pp), EAAG learns aggressive triggering (RR=73%). In rollout-harmful environments (Plancraft, Δ=−7.0 pp), trigger rate decays over steps (49% at step 0 → <20% later), becoming increasingly conservative as evidence of rollout harm accumulates. This full range, from "always trigger" to "almost never trigger," emerges from direction learning alone without explicit headroom estimation.

5.3 Ablation Studies

The main results show *that* EAAG works; we now ask *why*. We ablate three components in decreasing order of expected importance: direction learning, multi-signal features, and LLM reasoning.

Direction is the primary determinant of gate quality. Intentionally reversing the learned direction (BSW ablation) causes catastrophic failure across environments, with an MLP gate using the wrong direction falling *below* the no-trigger baseline. This confirms the central prediction of Proposition 1: direction is not a minor calibration detail but the primary determinant of gate quality. The magnitude of degradation correlates with signal strength (Appendix C), as the Two-Source Model predicts.

LLM reasoning provides robustness, not marginal accuracy. Removing the LLM reasoning step changes SR by <1 pp in the majority of environments. Two environments show larger effects: one where LLM-generated features capture step-0 criticality, and one where LLM features introduce noise. The LLM's value is not marginal accuracy improvement but *generalizability*: it enables zero-shot feature generation for unseen environments where task-specific signals cannot be anticipated by universal features alone.

[FIGURE NEEDED: Figure 6.] Environment-adaptive trigger rates. Bar chart: x-axis = 8 environments (ordered by rollout headroom Δ), y-axis = EAAG trigger rate (%). Show how trigger rate naturally correlates with headroom: high in TWExpress/HotpotQA, moderate in APPS, low

in WebShop, near-zero in Plancraft. No explicit headroom estimation needed. Similar to FLARE Figure 2 which shows dynamics across environments.

Gating magnitude emerges from direction learning alone. The learned gate adapts trigger rate across environments without explicit headroom estimation: aggressive triggering in rollout-safe environments, moderate in mixed environments, and step-dependent decay in rollout-harmful ones. These environment-adaptive gating patterns arise from simple logistic regression; the LASSO coefficients implicitly encode environment-specific triggering patterns through the learned signal–utility relationship.

5.4 Two-Source Model Verification

The ablations above confirm EAAG's practical advantage; we now test whether the *theoretical mechanism*, the Two-Source Model (§3.2), correctly predicts the observed patterns. Following Schaeffer et al. (2023), we derive three testable predictions from the model and verify each against held-out data:

- **P1: Temporal dynamics.** Within the same environment, ρ should *decrease* over the episode: early steps mix Type I and Type D uncertainty, while late steps, after information-gathering attempts, isolate the residual Type I component.
- **P2: Cross-environment consistency.** Environments with similar task structure should exhibit similar ρ (e.g., FEVER $\approx$ HotpotQA, both search-based QA).
- **P3: Signal identity alignment.** In Type I environments, the strongest signal should measure information sufficiency; in Type D environments, decision complexity.

P1: Temporal dynamics (confirmed). We split trajectory data into early (step $\leq$ median) and late (step $>$ median) subsets. In HotpotQA, ρ shifts from -0.176 (early) to -0.437 (late): late-step entropy reflects deeper information poverty after failed evidence retrieval. In APPS Intro, ρ shifts from $+0.102$ (early, non-significant) to -0.144 (late, p=0.04): early exploration diversity gives way to debugging frustration. In WebShop, the strong positive early-phase signal (ρ=$+0.285$) vanishes in the late phase. The pattern is consistent across all environments with sufficient signal (full data in Appendix D): early steps mix both uncertainty sources, while late steps isolate the dominant type.

P2: Cross-environment consistency (confirmed). We compare same-family pairs. FEVER (ρ=-0.119) and HotpotQA (ρ=-0.041), both search-based QA tasks requiring evidence retrieval, share negative entropy–utility direction. APPS Intro (ρ=$+0.012$) and APPS Interview (ρ=$+0.317$), both code generation tasks, share non-negative direction. The magnitude difference within each pair reflects the degree of Type I contamination; FEVER has more early-step information poverty than HotpotQA.

P3: Signal identity alignment (confirmed). We examine the LASSO-selected features per environment. In Type I environments (HotpotQA, FEVER), the strongest signal (`step_count`, $|\rho|$=0.494 and 0.619) measures trajectory progress, a proxy for information accumulation. In WebShop (mixed, choice-dominated), `num_available_actions` (ρ=$+0.444$) measures the decision space size, a proxy for decision complexity. All three predictions are confirmed, providing converging evidence for the Two-Source Model.

As a supplementary test, we manipulate information availability within HotpotQA itself (InfoPoor vs. InfoRich conditions) and confirm that the identity of the most informative signal shifts accordingly, supporting Observation 2. Full results are reported in Appendix F.3.

5.5 Robustness of Direction Reversal

A natural concern is whether the observed direction reversal is a genuine phenomenon or an artifact of confounders. We address this through three complementary analyses, plus multi-backbone verification (Appendix E) showing that direction depends on the *environment* $\times$ *model* interaction, further ruling out model-specific artifacts.

Stratified analysis: controlling for trajectory length. Trajectory length varies across environments and correlates with both entropy and utility. To rule out length as a confounder, we stratify by step count and recompute $\rho(\text{entropy}, U)$ within each stratum. *Result*: Direction reversal persists within trajectory-length strata where utility variance is non-zero. In HotpotQA, ρ remains negative across all three strata (-0.18/-0.46/-0.42). The reversal is not an artifact of trajectory length.

Table 5: Gate complexity ablation on HotpotQA. Direction matters more than model capacity: a logistic gate with correct direction outperforms an MLP with wrong direction by 49.9 pp.

Gate	Direction	SR (%)	Δ
Logistic (5 feat)	Correct	95.2	+46.2
MLP (5 feat)	Correct	∼95	∼+46
Hidden state (2560-d)	Correct	∼95	∼+46
Logistic (5 feat)	Wrong	62.0	+13.0
MLP (5 feat)	Wrong	45.3	−3.7
No gate (base)	—	49.0	0

Interventional evidence. The BSW ablation (§5.3) provides *interventional* evidence: reversing the gate's learned direction causes catastrophic SR degradation across environments, with the MLP gate falling below the no-trigger baseline. If direction reversal were merely correlational, this manipulation should have no systematic effect. The observed damage confirms that direction encodes a causal relationship.

Gate complexity ablation: direction ≫ capacity. To verify that the bottleneck is direction rather than gate complexity, we compare gate architectures with correct vs. wrong direction (Table 5).

With correct direction, all architectures achieve ∼95% SR; the jump from logistic to MLP to hidden-state probe adds <1 pp. With wrong direction, higher capacity makes performance *worse* (MLP 45.3% < logistic 62.0%), because the more powerful gate more precisely targets harmful states. The information hierarchy is: **signal direction ≫ signal count ≫ gate complexity**.

6 Limitations

EAAG underperforms on FEVER, where rollout value concentrates in the first one or two steps and random exploration misses these critical steps with 50% probability. This failure mode is itself predicted by the Two-Source Model: FEVER is the most extreme Type I environment, where undirected exploration is least efficient. Addressing this through curriculum-based exploration is an important direction for future work. We verify our findings on three backbones (4B–8B) across all eight environments (Appendix E); testing on larger models (70B+) remains future work. The Two-Source Model assumes linear conditional relationships and serves as an explanatory framework rather than a precise predictor of ρ values. EAAG requires 50 exploration episodes per new environment, a modest but non-zero cost amortized over subsequent deployment.

7 Conclusion

All existing adaptive gating methods for LLM agents assume that uncertainty signals have a fixed relationship with compute value. We show this assumption fails: signal–utility direction reverses across both environments and model backbones. A two-source uncertainty model grounded in Simpson's paradox explains when and why this occurs, and a necessity proof establishes that direction discovery is required for cross-environment non-negative value of computation. EAAG instantiates this principle through exploration, LLM-based reasoning, and sparse direction learning. Across eight environments and three model backbones, EAAG dominates all fixed-direction baselines, with adaptive gating behavior emerging from direction learning alone.

References

Yasin Abbasi Yadkori, Ilja Kuzborskij, and András György. 2024. To believe or not to believe your LLM. *arXiv preprint arXiv:2406.02543*.

Pranjal Aggarwal and Sean Welleck. 2025. L1: controlling how long A reasoning model thinks with reinforcement learning. *CoRR*.

Armen Der Kiureghian and Ove Ditlevsen. 2009. Aleatory or epistemic? does it matter? *Structural safety*.

Guanting Dong, Hangyu Mao, Kai Ma, Licheng Bao, Yifei Chen, Zhongyuan Wang, Zhongxia Chen, Jiazhen Du, Huiyang Wang, Fuzheng Zhang, Guorui Zhou, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. 2025. Agentic reinforced policy optimization. *CoRR*.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A Aly, Beidi Chen, and Carole-Jean Wu. 2024. Layerskip: Enabling early exit inference and self-speculative decoding. In *ACL*.

Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Thinkless: LLM learns when to think. *CoRR*.

Alex Graves. 2016. Adaptive computation time for recurrent neural networks. *CoRR*.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. Token-budget-aware LLM reasoning. In *ACL*.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. In *EMNLP*.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021a. Measuring coding challenge competence with APPS. In *NeurIPs*.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Andy Zou, Dawn Song, and Jacob Steinhardt. 2021b. Measuring coding challenge competence with APPS. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*.

Juyeon Heo, Miao Xiong, Christina Heinze-Deml, and Jaya Narain. 2025. Do llms estimate uncertainty well in instruction-following? In *ICLR*.

Eric Horvitz. 1987. Reasoning about beliefs and actions under computational resource constraints. In *UAI*.

Chengsong Huang, Langlin Huang, Jixuan Leng, Jiacheng Liu, and Jiaxin Huang. 2026. Cats: Calibrated test-time scaling for efficient llm reasoning. In *ICLR*.

Peter Jansen and Marc-Alexandre Côté. 2023. Textworldexpress: Simulating text games at one million steps per second. In *EACL*.

Chen Jin, Ryutaro Tanno, Tom Diethe, and Philip Teare. 2026. Corefine: Confidence-guided self-refinement for adaptive test-time compute. *CoRR*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, and 17 others. 2022. Language models (mostly) know what they know. *CoRR*.

Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2026. Agentic test-time scaling for webagents. *CoRR*.

Sungjae Lee, Hyejin Park, Jaechang Kim, and Jungseul Ok. 2025. Semantic exploration with adaptive gating for efficient problem solving with language models. In *ACL*.

Falk Lieder and Thomas L Griffiths. 2020. Resource-rational analysis: Understanding human cognition as the optimal use of limited computational resources. *Behavioral and brain sciences*.

Xiang Liu, Xuming Hu, Xiaowen Chu, and Eunsol Choi. 2025. Diffadapt: Difficulty-adaptive reasoning for token-efficient LLM inference. *CoRR*.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel J. Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. In *EMNLP*.

Davide Paglieri, Bartlomiej Cupial, Jonathan Cook, Ulyana Piterbarg, Jens Tuyls, Edward Grefenstette, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. 2025. Learning when to plan: Efficiently allocating test-time compute for LLM agents. *CoRR*.

Judea Pearl. 2022. Comment: Understanding simpson's paradox. In *Probabilistic and Causal Inference: The Works of Judea Pearl*.

David Raposo, Samuel Ritter, Blake A. Richards, Timothy P. Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *CoRR*.

Stuart J. Russell and Eric Wefald. 1991. *Do the right thing - studies in limited rationality*. MIT Press.

Nicolò De Sabbata, Theodore R. Sumers, and Thomas L. Griffiths. 2024. Rational metareasoning for large language models. *CoRR*.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are emergent abilities of large language models a mirage? In *NeurIPs*.

Aman Sharma and Paras Chopra. 2025. Think just enough: Sequence-level entropy as a confidence signal for LLM reasoning. *CoRR*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *NeurIPs*.

Edward H Simpson. 1951. The interpretation of interaction in contingency tables. *Journal of the Royal Statistical Society: Series B (Methodological)*.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *CoRR*.

Linwei Tao, Yi-Fan Yeh, Minjing Dong, Tao Huang, Philip Torr, and Chang Xu. 2025. Revisiting uncertainty estimation and calibration of large language models. *CoRR*.

Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. 2017. Branchynet: Fast inference via early exiting from deep neural networks. *CoRR*.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. Fever: a large-scale dataset for fact extraction and verification. In *NAACL*, pages 809–819.

Jingwei Wang, Jianshan He, Weidi Xu, Ruopeng Li, and Wei Chu. 2023. Learning to discover various simpson's paradoxes. In *KDD*.

Zehong Wang, Fang Wu, Hongru Wang, Xiangru Tang, Bolian Li, Zhenfei Yin, Yijun Ma, Yiyang Li, Weixiang Sun, Xiusi Chen, and Yanfang Ye. 2026. Why reasoning fails to plan: A planning-centric analysis of long-horizon decision making in LLM agents. *CoRR*.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *EMNLP*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPs*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In *ICLR*.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025. Adaptthink: Reasoning models can learn when to think. In *EMNLP*.

Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. 2026. Agentic uncertainty quantification. *CoRR*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language agent tree search unifies reasoning, acting, and planning in language models. In *ICML*.

Yubo Zhu, Dongrui Liu, Zecheng Lin, Wei Tong, Sheng Zhong, and Jing Shao. 2025. The LLM already knows: Estimating llm-perceived question difficulty via hidden representations. In *EMNLP*.

A Extended Related Work

A.1 Adaptive Compute: Reasoning vs. Agent Settings

The adaptive test-time compute literature spans two distinct settings. In the *reasoning* setting, methods optimize single-turn chain-of-thought generation on homogeneous benchmarks (MATH, GSM8K, GPQA). Signal-based approaches use token-level confidence or entropy to decide when to extend thinking (Lee et al., 2025; Huang et al., 2026; Jin et al., 2026; Sharma and Chopra, 2025; Muennighoff et al., 2025; Han et al., 2025). RL-based methods learn think/no-think policies through reinforcement learning (Zhang et al., 2025; Fang et al., 2025; Aggarwal and Welleck, 2025). Hidden-state probing methods train lightweight classifiers on internal representations to estimate task difficulty (Liu et al., 2025; Zhu et al., 2025). Because reasoning benchmarks share similar task semantics, signal direction happens to be consistent across tasks, and the fixed-direction assumption is never tested.

In the *agent* setting, methods operate in multi-step interactive environments with external optimizers. Vote-based methods (Lee et al., 2026) sample multiple candidate actions and trigger evaluation when disagreement is high. Paglieri et al. (2025) learn when to invoke a planner at the episode level. Agentic RL methods (Dong et al., 2025) use entropy-based adaptive rollout at tool-call steps. All agent-setting methods inherit the fixed-direction assumption from the reasoning literature without verifying it. Our work is the first to systematically study signal semantics across heterogeneous agent environments, where we find that the direction reverses.

A.2 Detailed Baseline Comparison

Table 6 provides a detailed comparison of all baselines evaluated in this paper. All six methods share a common assumption: a fixed mapping from signal value to compute need.

A.3 Adaptive Computation in Neural Networks

The idea of adapting computation to input difficulty predates LLMs. Adaptive Computation Time (Graves, 2016) allows RNNs to learn how many computational steps to take per input. Early exit methods (Teerapittayanon et al., 2017; Elhoushi et al., 2024) attach classifiers at intermediate transformer layers and exit early for easy inputs. Mixture-of-Experts architectures (Shazeer et al., 2017; Raposo et al., 2024) activate only a subset of parameters per token, adapting capacity rather than depth. These methods adapt computation within a single forward pass. Our work operates at a different granularity: we adapt whether to invoke an external optimizer at each decision step, and the key challenge is not input difficulty but the *direction* of the signal–utility relationship.

Table 6: Detailed comparison of baselines. All methods assume a fixed signal–utility direction. Phase 1 methods require a separate calibration dataset; EAAG requires only exploration episodes.

Method	Signal	Direction	Granularity	Mechanism
CATTS	Vote entropy + margin	High disagree. → trigger	Per-step	K=5 forward passes, arbiter on disagreement
SEAG†	Mean token confidence	Low conf. → search	Per-problem	Platt scaling, search depth allocation
CaTS†	Calibrated confidence	Low conf. → generate	Per-attempt	Self-calibration, Best-of-N early stopping
CoRefine†	Conv1D confidence	Low conf. → refine	Per-problem	Halt / re-examine / redirect controller
s1_budget	Token budget	Fixed budget → stop	Per-problem	Budget-constrained generation
AUQ	Verbalized uncertainty	High uncert. → reflect	Per-step	Uncertainty-aware memory + reflection (Zhang et al., 2026)
EAAG	**Multi (auto)**	**Learned**	**Per-step**	Explore + calibrate + LASSO direction learning

A.4 LLM Confidence Estimation and Calibration

Kadavath et al. (2022) show that larger models are well-calibrated on multiple-choice tasks and can estimate P(True) for their own outputs. Tao et al. (2025) and Heo et al. (2025) find that calibration quality varies substantially across task types, providing converging evidence that uncertainty semantics are domain-dependent. Abbasi Yadkori et al. (2024) propose decoupling epistemic and aleatoric uncertainty in LLM outputs, a distinction conceptually related to our Type I/Type D decomposition. Our work extends these findings: not only does calibration *quality* vary across tasks, but the *direction* of the uncertainty–utility relationship reverses entirely.

A.5 Metareasoning and Value of Computation

Rational metareasoning (Russell and Wefald, 1991; Horvitz, 1987) formalizes the value of computation (VOC) as a guide for allocating cognitive resources. The classical framework assumes VOC ≥ 0 because an agent can always ignore computation results. Lieder and Griffiths (2020) generalize this to resource-rational analysis, modeling cognition as optimal use of limited computational resources. Sabbata et al. (2024) apply metareasoning to LLMs, analyzing when to extend chain-of-thought reasoning. Our work extends this framework by showing that VOC can be *negative* in the agent setting when the evaluator and executor are the same model. In this regime, wrong-direction gating does not merely waste computation (VOC $= 0$) but actively degrades performance (VOC $\ll 0$), violating the classical assumption. Direction discovery is therefore a necessary precondition for non-negative VOC across environments.

A.6 Simpson's Paradox in Machine Learning

Simpson's paradox (Simpson, 1951; Pearl, 2022) occurs when a trend present in subgroups reverses upon aggregation. In machine learning, this phenomenon has been documented in fairness (Wang et al., 2023), causal inference, and treatment effect estimation. Our Two-Source Model identifies a new instance: within Type I states, entropy negatively predicts utility; within Type D states, entropy positively predicts utility; but the marginal correlation depends on the mixture proportion $p_I(\mathcal{E})$, which varies across environments. This makes direction reversal a structural consequence of aggregating heterogeneous uncertainty sources.

A.7 Concurrent Work Statement

Several 2025–2026 papers independently explore adaptive compute. CATTS (Lee et al., 2026) (Feb 2026) and CoRefine (Jin et al., 2026) (Feb 2026) are concurrent with no methodological overlap. AdaptThink (Zhang et al., 2025) (May 2025) and Thinkless (Fang et al., 2025) (May 2025) predate our work but operate in the reasoning setting. DiffAdapt (Liu et al., 2025) (Oct 2025) assumes a universal U-shaped entropy pattern that our cross-environment evidence disproves. AUQ (Zhang et al., 2026) (Jan 2026) uses verbalized uncertainty for agent reflection but assumes fixed direction. Our distinct contribution is threefold: first to (a) demonstrate direction reversal across environments and backbones, (b) formalize the mechanism via a Two-Source Model grounded in Simpson's paradox, and (c) prove that direction discovery is a necessary condition for cross-environment non-negative VOC.

B Experimental Details

B.1 Environment Specifications

For each of the 8 environments, we provide:

- Task description and action space

- State representation and reward function
- Episode length distribution
- Optimizer T implementation details:
  - HotpotQA, FEVER, TWExpress, Plancraft: per-action evaluation (generate K=5 candidates, evaluate each, select best)
  - APPS Intro, APPS Interview, CRUXEval: K-variant sampling (generate K=3 code solutions, test each, select passing one)
  - WebShop: LLM-Propose-K (LLM proposes K=3 action sequences, simulate each, select highest-reward)

B.2 Hyperparameter Configuration

EAAG hyperparameters. $N_{\text{explore}} = 50$ episodes, $\varepsilon_{\text{explore}} = 0.5$, $\varepsilon_{\text{deploy}} = 0.1 \rightarrow 0$ over 100 episodes, LASSO α selected via 5-fold CV, gate threshold $\tau = 0.5$, retrain interval = 30 episodes.

Baseline hyperparameters. CaTS: Platt scaling on Phase 1 data (200 episodes), threshold = 0.5. SEAG: mean confidence threshold calibrated on Phase 1. CoRefine: entropy threshold calibrated on Phase 1. CATTS: K=5 votes, entropy threshold = 0.8, margin threshold = 0.2. BSW: EAAG with intentionally reversed direction (sign flip).

B.3 Feature Selection Details

[DATA NEEDED: Table A1.] Environment × Feature matrix showing LASSO coefficients. Rows = 8 environments. Columns = step_count, token_entropy, state_length, action_entropy, num_available_actions, has_output, + LLM-generated features per env. Values = LASSO coefficient (0 = not selected, sign = direction). This is the detailed version of Figure 2 in the main text.

[DATA NEEDED: Table A2.] LLM-generated features per environment. Columns: Environment, LLM Prompt (abbreviated), Generated Features, LASSO Selected (Y/N), Coefficient. Shows the LLM reasoning step's concrete output for each environment.

B.4 Computational Cost Breakdown

[DATA NEEDED: Table A3.] Method × Cost component. Columns: Method, Phase-1 Episodes, Phase-1 Compute (GPU-hrs), Per-Step Overhead (ms), Gate Training Time, Total Cost for 500 Episodes. Shows EAAG's cost advantage: 50 exploration episodes, ∼2 GPU-hrs, 0ms gate overhead, <1s training.

C Proofs

C.1 Proof of Proposition 1 (Necessity of Direction Discovery)

Proposition (Restated)**.** *Suppose there exist two environments $\mathcal{E}_1, \mathcal{E}_2$ with opposite true directions: $d^*(\mathcal{E}_1) = +1$ and $d^*(\mathcal{E}_2) = -1$. Then no fixed-direction gate g_d can simultaneously satisfy $\text{SR}(g_d, \mathcal{E}_1) \geq \text{SR}(\text{base}, \mathcal{E}_1)$ and $\text{SR}(g_d, \mathcal{E}_2) \geq \text{SR}(\text{base}, \mathcal{E}_2)$.*

Proof. Define a fixed-direction gate as $g_d(s) = \mathbf{1}[d \cdot \sigma(s) > \theta]$ for $d \in \{+1, -1\}$ and threshold θ.

Define $d^*(\mathcal{E}) = +1$ if $\text{Corr}(\sigma(s), U(T, s)) > 0$ in $\mathcal{E}$ (Type D), and $d^*(\mathcal{E}) = -1$ if $\text{Corr}(\sigma(s), U(T, s)) < 0$ (Type I).

Lemma (Wrong-direction harm). If $d \neq d^*(\mathcal{E})$, then $\text{SR}(g_d, \mathcal{E}) \leq \text{SR}(\text{base}, \mathcal{E}) - \delta(\mathcal{E})$ where $\delta(\mathcal{E}) > 0$ depends on signal strength $|\rho(\mathcal{E})|$.

Proof of Lemma: By definition of d^*, states with high $d \cdot \sigma(s)$ are states where $U < 0$. The gate triggers T at these states, causing the agent to adopt worse actions. The base policy, which never triggers T, avoids this systematic harm.

Main argument:

Case 1: $d = +1$. In $\mathcal{E}_2$ where $d^* = -1$, $d \neq d^*(\mathcal{E}_2)$. By Lemma: $\text{SR}(g_{+1}, \mathcal{E}_2) \leq \text{SR}(\text{base}, \mathcal{E}_2) - \delta(\mathcal{E}_2) < \text{SR}(\text{base}, \mathcal{E}_2)$. Constraint violated.

Case 2: $d = -1$. In $\mathcal{E}_1$ where $d^* = +1$, by symmetric argument, constraint violated.

Since $d \in \{+1, -1\}$ and both cases lead to violation, no fixed d satisfies both constraints simultaneously.

Empirical grounding: $\delta(\text{HotpotQA}) \approx 37.0$ pp (BSW ablation), $\delta(\text{WebShop}) \approx 23.2$ pp, $\delta(\text{FEVER}) \approx 36.8$ pp. MLP with wrong direction: 45.3% < base 49.0% on HotpotQA. □

C.2 Wrong-Direction Damage Quantification

Table 7: Wrong-direction damage scales with signal strength.

Environment	Correct SR	Wrong SR	Δ SR	$\lvert\rho\rvert$ (strongest)
HotpotQA	95.2%	58.2%	−37.0	0.494
WebShop	43.8%	21.4%	−22.4	0.444
FEVER	49.8%	13.0%	−36.8	0.619
APPS Intro	66.0%	62.5%	−3.5	0.155

The damage magnitude scales with signal strength: environments with stronger signal–utility coupling ($|\rho|$) suffer larger SR degradation when direction is reversed ($R^2 > 0.5$). On HotpotQA, the MLP gate with wrong direction achieves 45.3%, *below* the 49.0% base, demonstrating that a more powerful gate with wrong direction is *worse* than a simpler gate with wrong direction (logistic: 62.0%), because it more precisely targets harmful states.

C.3 VOC Non-Negativity Scope

In the classical VOC framework (Russell and Wefald, 1991), $\mathrm{VOC}(T, s) = \mathbb{E}[\max(V(a_T), V(a_{\text{base}}))] - V(a_{\text{base}}) \geq 0$ because the agent can *disregard* the computation result if unfavorable. Our setting violates this: when the agent uses an evaluator/optimizer T, the evaluator's assessment *is* the agent's decision. The agent cannot separately evaluate the evaluator's output because the evaluator *is* the evaluation mechanism.

Under evaluator-executor identity, VOC can be negative: $\mathrm{VOC}(T, s) = \mathbb{E}[V(a_T(s))] - V(a_{\text{base}}(s))$. If T systematically selects worse actions in Type I states, $\mathrm{VOC} < 0$ for those states. This explains why wrong-direction gating is not merely wasteful ($\mathrm{VOC} = 0$) but actively harmful ($\mathrm{VOC} < 0$).

D Two-Source Model: Full Derivation and Verification

D.1 Full Derivation

At each step t, state s_t belongs to one of two latent types: $s_t \in$ Type I with probability $p_I(\mathcal{E})$, $s_t \in$ Type D with probability $1 - p_I(\mathcal{E})$.

Conditional utility models:

$$U(T, s) \mid s \in \text{Type I} = -\alpha \cdot H(s) + \varepsilon_I, \quad \alpha > 0 \tag{4}$$

$$U(T, s) \mid s \in \text{Type D} = +\beta \cdot H(s) + \varepsilon_D, \quad \beta > 0 \tag{5}$$

The marginal correlation:

$$\begin{aligned} \mathrm{sign}(\rho(\mathcal{E})) &= \mathrm{sign}(\beta \cdot (1 - p_I) - \alpha \cdot p_I) && (6) \\ &= \mathrm{sign}(\beta - (\alpha + \beta) \cdot p_I) && (7) \end{aligned}$$

Reversal threshold: $p_I^* = \beta/(\alpha + \beta)$.

D.2 Environment Mapping on the p_I Axis

[FIGURE NEEDED: Figure A1.] 1D axis diagram. Horizontal axis from $p_I = 0$ (pure Type D) to $p_I = 1$ (pure Type I). Mark each environment's estimated position with a labeled point. Show the reversal threshold p_I^* as a vertical dashed line. Left of p_I^*: positive ρ (green). Right: negative ρ (red). Provides a visual summary of the entire Two-Source Model.

Estimated positions on the p_I axis (0 = pure Type D, 1 = pure Type I):

- $p_I \approx 0.2$: APPS Interview (strong positive $\rho = +0.317$)
- $p_I \approx 0.45$: APPS Intro (near-zero $\rho = +0.012$)
- $p_I \approx 0.5$: WebShop (entropy $\rho \approx 0$, but non-entropy signals work)
- $p_I \approx 0.55$: CRUXEval (weak negative $\rho = -0.048$)
- $p_I \approx 0.6$: Plancraft (weak negative, rollouts harmful)
- $p_I \approx 0.7$: HotpotQA (negative $\rho = -0.041$)
- $p_I \approx 0.8$: TWExpress (negative $\rho = -0.290$)
- $p_I \approx 0.9$: FEVER (strong negative $\rho = -0.119$)

D.3 Prediction Verification: Full Data

[DATA NEEDED: Table A4.] P1 Temporal Dynamics full data. Columns: Environment, ρ_{early} (step $\leq$ median), ρ_{late} (step $>$ median), p_{early}, p_{late}. All 8 environments. Shows the temporal shift pattern for each.

[DATA NEEDED: Table A5.] P3 Signal Identity Alignment. Columns: Environment, Dominant Type, Strongest Signal, $|\rho|$, Signal Interpretation. Shows what each environment's strongest signal measures and how it aligns with the predicted type.

E Multi-Backbone Verification

We validate our core findings on three backbones from three vendors: Qwen3-4B-Instruct (Alibaba, 4B), Phi-3.5-mini-instruct (Microsoft, 3.8B), and Llama-3.1-8B-Instruct (Meta, 8B) across all 8 environments.

E.1 Signal Direction: Environment $\times$ Model Interaction

The multi-backbone results (Table 1 in main text) reveal that 4/5 environments with $\geq$2 significant signals show cross-model direction variation. Structural signals (`step_count`) are

model-invariant while entropy signals are model-dependent.

[DATA NEEDED: Table A6.] Full multi-backbone results. 8 environments $\times$ 3 backbones $\times$ multiple signals. Extends Table 1 in main text with all signal types, not just entropy. Shows that step_count direction is stable while entropy flips.

E.2 Fixed-Direction Baselines Fail Across Backbones

On Phi-3.5 APPS: all 6 CB methods score *below* base_only (59% $\rightarrow$ 27–36%), demonstrating that fixed-direction assumption is catastrophically wrong when the model changes.

[DATA NEEDED: Table A7.] Cross-backbone baseline results. Methods $\times$ environments for Phi-3.5 and Llama-3.1 backbones. Shows that the same baselines that work on Qwen3 fail on other backbones, while EAAG adapts successfully.

F Additional Analyses

F.1 Hyperparameter Sensitivity

[FIGURE NEEDED: Figure A2.] Hyperparameter sensitivity plots. (a) SR vs $N_{\text{explore}} \in \{10, 20, 30, 50, 100\}$ for 4 environments. Shows stability for $N_{\text{explore}} \geq 30$. (b) SR vs gate threshold $\tau \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$. Shows robustness across threshold range. Similar to standard ablation sensitivity plots in NeurIPS papers.

EAAG is robust to $N_{\text{explore}} \in [30, 100]$ (stable SR for $N_{\text{explore}} \geq 30$; marginal gains above 50) and gate threshold $\tau \in [0.3, 0.7]$.

F.2 Statistical Significance

All head-to-head comparisons use 3-seed evaluation (200 episodes/seed). Bootstrap confidence intervals (5000 resamples) are reported for all comparisons. 18 of 34 wins are statistically significant (95% CI excludes zero). Pareto-dominance claims use a conservative criterion that does not rely on p-values.

[DATA NEEDED: Table A8.] Full statistical significance table. For each method $\times$ environment pair: ΔSR, 95% bootstrap CI, significant (Y/N). 48 cells total (6 baselines $\times$ 8 environments).

F.3 Controlled Information Manipulation: Full Results

To provide stronger evidence that information structure drives signal semantics, we manipulate information availability within HotpotQA itself. *InfoPoor* (search returns only the first sentence) reduces base SR to 44.0% while *InfoRich* (gold evidence injected at start) raises it to 82.0%. The signal hierarchy shifts: in InfoPoor, `step_count` dominates (ρ=-0.608) while entropy carries a weak positive signal (ρ=$+0.119$, n=317; note that the Two-Source Model predicts entropy should be negative in info-poor settings, but the small sample and mixed early-step Type D component may explain the weakly positive sign; the key finding is that `step_count` overtakes entropy as the dominant signal); in InfoRich, entropy becomes the dominant positive signal (ρ=$+0.311$) while `step_count` weakens (ρ=-0.147). This demonstrates that the *identity* of the most informative signal, not just its direction, is a function of the environment's information structure, supporting Observation 2.

[DATA NEEDED: Table A9.] InfoPoor vs InfoRich full signal correlation table for all features. Columns: Condition, Base SR, ρ for each feature, strongest signal.

F.4 LLM Reasoning Prompts and Outputs

[DATA NEEDED.] Full LLM prompt template used in the Reason step of EAAG. Include one complete example showing: (1) the prompt given to the LLM with exploration data summary, (2) the LLM's full output including environment profile and feature hypotheses, (3) which features LASSO subsequently selected. Use WebShop as the example (most interesting LLM contribution). This is important for reproducibility (NeurIPS checklist item).

G Broader Impact

This work addresses computational efficiency in LLM agent deployment. By reducing unnecessary optimizer invocations, EAAG lowers the energy and cost footprint of test-time compute without sacrificing task performance. The prescriptive framework (characterize information structure before designing gates) may help practitioners avoid deploying fixed-direction methods that actively harm performance in mismatched environments. We do not foresee direct negative societal impacts beyond those inherent to the LLM agents themselves.